`CrudRepository` for `Book` objects are based on their `@Id`-annotated fields. Since the application needs to access books based on the ISBN, we must explicitly declare those operations.

Listing 5.13 Repository interface for accessing books

```
package com.polarbookshop.catalogservice.domain;

import java.util.Optional;
import org.springframework.data.jdbc.repository.query.Modifying;
import org.springframework.data.jdbc.repository.query.Query;
import org.springframework.data.repository.CrudRepository;

public interface BookRepository
    extends CrudRepository<Book,Long> {                    ❶

  Optional<Book> findByIsbn(String isbn);                  ❷
  boolean existsByIsbn(String isbn);

  @Modifying                                               ❸
  @Query("delete from Book where isbn = :isbn")            ❹
  void deleteByIsbn(String isbn);
}
```

❶ Extends a repository providing CRUD operations, specifying the type of managed entity (Book) and its primary key type (Long)

❷ Methods implemented by Spring Data at runtime

❸ Identifies an operation that will modify the database state

❹ Declares the query that Spring Data will use to implement the method

At startup time, Spring Data will provide an implementation for `BookRepository` with all the most common CRUD operations and the methods you declared in the interface. There are two main options for defining custom queries in Spring Data:

- Using the `@Query` annotation to provide an SQL-like statement that will be executed by the method.